Course : Python Programming



Ms. Sushama Deshmukh

Assistant Professor in Computer Science & Engineering

Course Instructor

© Copyright | Sushama Deshmukh

Course Structure

Teaching Scheme	Evaluation Scheme
Theory : 2 Hrs/week	CA: 60 Marks
Practical: 2 Hrs/week	End Sem: 40 Marks
Total credits: 3 (2+1)	
Course Objectives	1. To understand fundamental concepts in Python Programming 2. To learn the different Conditional Loops and Iteration 3. To understand various data structures and packages
Course Outcomes	**Student will able to,** **CO1:** Describe programming fundamentals of python. **CO2:** Interpret the python syntax and semantics of control flow statements. **CO3:** Identify the methods to create and manipulate programs with python data structures. **CO4:** Use modular approach for problem solving. **CO5:** Apply advanced features and packages of python programming required for data science.
Pre-requisites	1. Basic Computer Knowledge 2. Knowledge of any programming language(optional)

© Copyright | Sushama Deshmukh

Course Content

Unit	Content	Teaching hours
1	**Python for everybody** Why Program, Hardware Overview, Python as a Language, Why Python Installation Python Jupyter notebook, Using the Python Playground, how to write program and compile in Jupyter notebook. Writing input and output statements in Python, output formatting, Writing comments keywords in Python .variables and variable assignments, Operators, writing expressions.	6
2	**Conditional Loops and Iteration** Conditional Statements, in Python, Loops and Iteration, Definite Loops, Finding the Largest Value, Loop Indioms.	6
3	**Data Structures in Python** Strings, Manipulating Strings, Files, Processing Files, Dictionaries, and sets, Tuples, Lists Manipulating Lists, Lists and Strings, Strings, Manipulating Strings.	6
4	**Functions, Modules and Packages** Functions, Lambda functions, Recursive function, Types of functions, modules and packages.	6
5	**Packages in Python for Data Science** Numpy introduction, Numerical operations on Numpy, Introduction of Matplotlib, getting started with Pandas, Data frames basics in Pandas, key operations on data frames. Introduction to Data Science, Binary search, finding elements in common in lists using Hash, Finding largest elements, Introduction of SQL	6

© Copyright | Sushama Deshmukh

Text, Reference Books & Web Links

Text Books:
1. D. Lee, "Python Programming Fundamentals", Second Edition, Springer Publication..
2. Wes McKinney, "Python for Data Analysis" O'Reilly Publication.

Reference Books: Reference Books:

1. The Python Language Reference: http://docs.python.org/2/reference/index.html
2. The Python Standard Library: http://docs.python.org/2/library/
3. https://docs.scipy.org/doc/scipy/reference/tutorial/stats.html
4. http://matplotlib.org/api/mlab_api.html#module-matplotlib.mlab
5. http://conference.scipy.org/proceedings/scipy2010/pdfs/seabold.pdf
6. http://seaborn.pydata.org
7. https://www.datacamp.com/community/data-science-cheatsheets
8. PEP 20 -- The Zen of Python: https://www.python.org/dev/peps/pep-0020/
9. https://docs.scipy.org/doc/numpy-dev/user/numpy-for-matlab-users.html

© Copyright | Sushama Deshmukh

Course Outcome

- Describe programming fundamentals of python
- Interpret the python syntax and semantics of control flow statements
- Identify the methods to create and manipulate programs with python data structures
- Use modular approach for problem solving
- Apply various features and packages of python programming required for data science.

© Copyright | Sushama Deshmukh

Unit I

Python for Everybody

Unit I - Learning Outcome

- Describe programming fundamentals of python.
- Demonstrate python program using development environment.

© Copyright | Sushama Deshmukh

Index

- Why Program?
- Hardware Overview
- Python as a Language, Why Python?
- Installation of Python Jupyter notebook, Using the Python Playground,
- How to write program and compile in Jupyter notebook.
- Writing input and output statements in Python,
- Output formatting, Writing comments,
- Keywords, Variables and variable assignments,
- Operators, writing expressions

© Copyright | Sushama Deshmukh



© Copyright | Sushama Deshmukh

What is a Language?



- A language is a medium of communication.
- Humans use different types of languages to communicate with each other.

  Ex: English, Hindi or their mother tongue.
- Human language is made up of characters, strings, numbers and symbols.

*** Computer can understand only the language of 0's and 1's.**

© Copyright | Sushama Deshmukh

Why Program? What is a Computer Language?



Ex: C , C++, Java, JSP, VB.net, Fortron, Cobol, …. etc.

- Computer(programming) language is **a mode of the communication** between **human, hardware components and computer.**
- **A programming language** is also made up of characters, variables, constants, digits and special symbols.
- **What is a Program?**

  It is a set of instructions.

© Copyright | Sushama Deshmukh

Hardware Overview

Computer Components

https://www.youtube.com/watch?v=OdziYWEkDIM

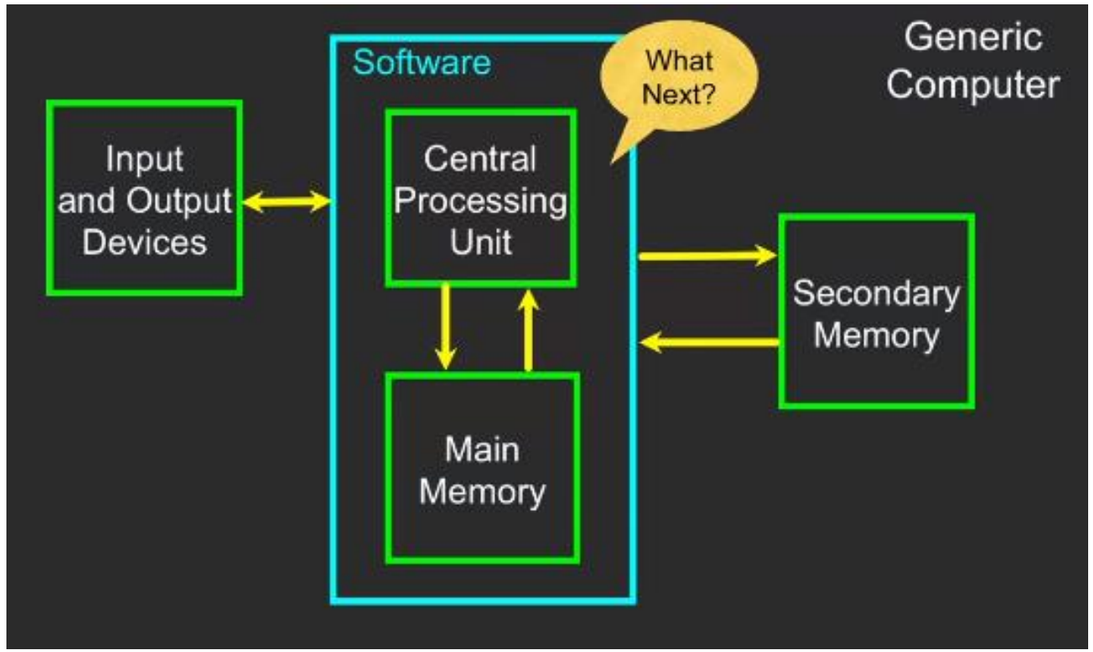
How does Computer Hardware Work?

https://www.youtube.com/watch?v=d86ws7mQYIg

How does Computer Memory Work?

https://www.youtube.com/watch?v=7J7X7aZvMXQ

https://www.youtube.com/watch?v=p3q5zWCw8J4



https://www.coursera.org/learn/python/lecture/zBbDN/1-2-hardware-overview

© Copyright | Sushama Deshmukh

Knowledge Check



© Copyright | Sushama Deshmukh

Knowledge Check



1. Program that give instructions to the computer are called as.....
2. Which hardware is called the brain of the computer?
3. Hardware can not perform any task without.....
4. A combination of ..... & ..... forms a usable computing system.
5. What is the name of this hardware?





© Copyright | Sushama Deshmukh

Why Python?

- Python is a popular programming language. It was created by **Guido van Rossum,** and released in 1991.
- Python works on different platforms.
- It has a simple syntax
- It has syntax that allows developers to write programs
- Python runs on an interpreter system.
- Python can be treated in a procedural way, an object-oriented way or a functional way.

https://medium.com/javarevisited/why-python-programming-language-is-important-in-data-science-beb4a7f91f75

© Copyright | Sushama Deshmukh

Applications of Python?

- It is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting..

https://medium.com/javarevisited/why-python-programming-language-is-important-in-data-science-beb4a7f91f75

© Copyright | Sushama Deshmukh

Features of Python?

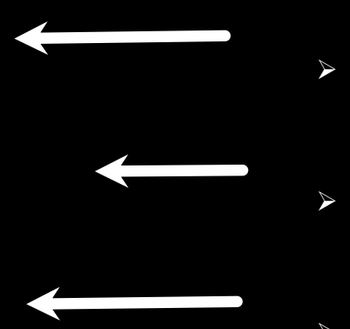
- Easy-to-learn
- Easy-to-read
- Easy-to-maintain
- A broad standard library
- Interactive Mode
- Portable
- Extendable
- Databases
- GUI Programming
- Scalable

https://medium.com/javarevisited/why-python-programming-language-is-important-in-data-science-beb4a7f91f75

© Copyright | Sushama Deshmukh

Sentences or Lines

```
x = 2
x = x + 2
print(x)
```



Assignment statement

Assignment with expression

Print statement

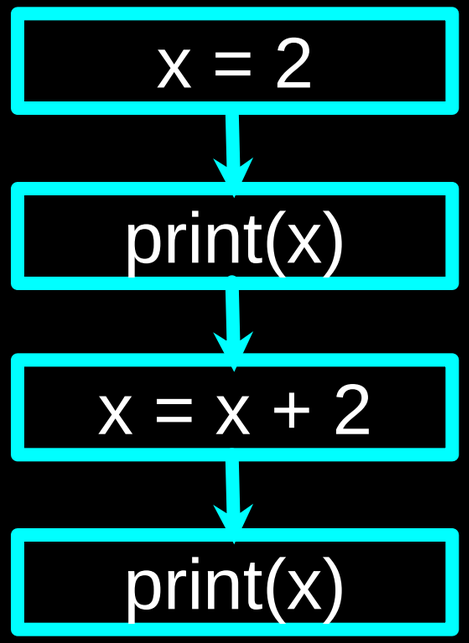
Variable

Operator

Constant

Function

Sequential Steps



Program:

```
x = 2
print(x)
x = x + 2
print(x)
```

Output:

```
2
4
```

When a program is running, it flows from one step to the next.
As programmers, we set up “paths” for the program to follow.

Writing input and output statements

➢ **How to Print Output in Python?**

- The **print()** function.
- This function allows us to display text, variables, and expressions on the console.
- Let's begin with the basic usage of the **print( )** function:

Example: Program code

print("Hello World")

Output:
Hello, World!

➢ **Why Single or Double Quotes?**

- String variables can be declared either by using single or double quotes:

Example:

print('Hello World')

© Copyright | Sushama Deshmukh

Writing input and output statements

- **Taking input in Python :**

- Python input() function is used to take user input. By default, it returns the user input in form of a string.
- **Syntax:**

  input(prompt)

Example: Program code

```
name = input("Enter your name: ")
print("Hello,", name, "! Welcome!")
```

Output:

Enter your name: Sushama
Hello, Sushama ! Welcome!

© Copyright | Sushama Deshmukh

Writing Comments

- Comments can be used **to explain Python code.**
- Comments can be used **to make the code more readable.**
- Comments can be used **to prevent execution when testing code.**

Creating a Comment

Comments starts with a #, and Python will ignore them.

Multiline comment:

For multiline comments **three times double quote or single quote** given **at the start & end** of the of the statement.

Example: Program code

```
#This is a comment
print("Hello, World!" )
```

Output:

print("Hello, World!") #This is a comment

© Copyright | Sushama Deshmukh

Keywords

- Python has a set of keywords that are reserved words that cannot be used as variable names, function names, or any other identifiers:

Keyword	Description
and	A logical operator
as	To create an alias
assert	For debugging
break	To break out of a loop
class	To define a class
continue	To continue to the next iteration of a loop
def	To define a function
del	To delete an object
elif	Used in conditional statements, same as else if
else	Used in conditional statements

© Copyright | Sushama Deshmukh

Keywords

except	Used with exceptions, what to do when an exception occurs
FALSE	Boolean value, result of comparison operations
finally	Used with exceptions, a block of code that will be executed no matter if there is an exception or not
for	To create a for loop
from	To import specific parts of a module
global	To declare a global variable
if	To make a conditional statement
import	To import a module
in	To check if a value is present in a list, tuple, etc.
is	To test if two variables are equal
lambda	To create an anonymous function
None	Represents a null value

© Copyright | Sushama Deshmukh

Keywords

nonlocal	To declare a non-local variable
not	A logical operator
or	A logical operator
pass	A null statement, a statement that will do nothing
raise	To raise an exception
return	To exit a function and return a value
TRUE	Boolean value, result of comparison operations
try	To make a try...except statement
while	To create a while loop
with	Used to simplify exception handling
yield	To return a list of values from a generator

© Copyright | Sushama Deshmukh

Variables and variable assignments

- Variables are containers for storing data values.

- **Creating Variables**

- Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.

Example:

```
x = 6
y = “Rajiv"
print(x)
print(y)
```


© Copyright | Sushama Deshmukh

Variables and variable assignments

- Variables do not need to be declared with any particular type, and can even change type after they have been set.

Example:

```
x = 8                  # x is of type int
x = “Saniya"       # x is now of type str
print(x)
```


© Copyright | Sushama Deshmukh

Variables and variable assignments

➢ **Casting**

If you want to specify the data type of a variable, this can be done with casting.

Example:

```
x = str(3)    # x will be '3'
y = int(3)    # y will be 3
z = float(3)  # z will be 3.0
```

➢ **Get the Type**

You can get the data type of a variable with the **type()** function.

Example:

```
x = 6
y = "Rajiv"
print(type(x))
print(type(y))
```


© Copyright | Sushama Deshmukh

Variables and variable assignments

- **Case-Sensitive**

  Variable names **are case-sensitive.**

Example:

```
a = 4
A = "Sally"
#A will not overwrite a
```

This will create two variables:

© Copyright | Sushama Deshmukh

Variables and variable assignments

➢ **Variable Names**

- A variable can have a short name (like x and y) or a more descriptive name. (age, carname, total_volume).

➢ **Rules for Python variables:**

- A variable name must **start with a letter or the underscore character**
- A variable name **cannot start with a number**
- A variable name **can only contain alpha-numeric characters and underscores** (A-z, 0-9, and _)
- Variable names **are case-sensitive** (age, Age and AGE are three different variables)
- A variable name **cannot be any of the Python keywords.**

Example:

```
x = 8                  # x is of type int
x = “Saniya"           # x is now of type str
print(x)
```


© Copyright | Sushama Deshmukh

Variables and variable assignments

- **Legal variable names:**

Example:

```
myvar = "John"
my_var = "John"
_my_var = "John"
myVar = "John"
MYVAR = "John"
myvar2 = "John"
```

- **Illegal variable names:**

Example:

```
2myvar = "John"
my-var = "John"
my var = "John"
```

*** Remember that variable names are case-sensitive**

© Copyright | Sushama Deshmukh

Variables and variable assignments

➢ **Multi Words Variable Names:**

Variable names with more than one word can be difficult to read.
There are several techniques you can use to make them more readable:

- **Camel Case:** Each word, except the first, starts with a capital letter:

Example: myVariableName = "John"

- **Pascal Case :** First letter of each word is in uppercase

Example: MyVariableName = "John"

© Copyright | Sushama Deshmukh

Variables and variable assignments

- **Multi Words Variable Names continued**

- **Snake Case:** Each word is separated by an underscore character:

Example: my_variable_name = "John"

© Copyright | Sushama Deshmukh

Variables and variable assignments

- **Assign Multiple Values:** Many Values to Multiple Variables
  - Python allows you to assign values to multiple variables in one line:

Example:

```
x, y, z = "Orange", "Banana", "Cherry"

print(x)

print(y)

print(z)
```

Note: Make sure the number of variables matches the number of values, or else you will get an error.

© Copyright | Sushama Deshmukh

Variables and variable assignments

- **One Value to Multiple Variables**

  - And you can assign the same value to multiple variables in one line:

Example:

```
x = y = z = "Orange"

print(x)

print(y)

print(z)
```


© Copyright | Sushama Deshmukh

Variables and variable assignments

- **Unpack a Collection**

  - If you have a collection of values in a list, tuple etc. Python allows you to extract the values into variables. This is called unpacking**.**

Example: Unpack a list:

```
fruits = ["apple", "banana", "cherry"]

x, y, z = fruits

print(x)

print(y)

print(z)
```


© Copyright | Sushama Deshmukh

Variables and variable assignments

➢ Output Variables

- The Python print( ) function is often used to output variables.

Example:

```
x = "Python is awesome"
print(x)
```

- In the print( ) function, you output multiple variables, separated by a comma:

Example:

```
x = "Python"
y = "is"
z = "awesome"
print(x, y, z)
```


© Copyright | Sushama Deshmukh

Variables and variable assignments

➢ Output Variables

- You can also use the + operator to output multiple variables:

Notice the space character after "Python " and "is ", without them the result would be "Pythonisawesome".

Example:

```
x = "Python "
y = "is "
z = "awesome"
print(x + y + z)
```

O/P:

Python is awesome

- For numbers, the + character works as a mathematical operator:

Example:

```
x = 5
y = 10
print(x + y)
```

O/P:

15

© Copyright | Sushama Deshmukh

Variables and variable assignments

➢ Output Variables

- In the print( ) function, when you try to combine a string and a number with the + operator, Python will give you an **error:**

Example:

```
x = 5
y = "John"
print(x + y)
```

- The best way to output multiple variables in the print( ) function is to separate them with commas, which even support different data types:

Example:

```
x = 5
y = "John"
print(x, y)
```

O/P:

5 John

© Copyright | Sushama Deshmukh

Variables and variable assignments

Example:

```
x = “awesome”

def myfunc( ):

  print("Python is " + x)

myfunc( )
```

Output:

Python is awesome

➢ **Global Variables**

- Variables that are created outside of a function (as in all of the examples above) are known as global variables.
- Global variables can be used by everyone, both inside of functions and outside.

© Copyright | Sushama Deshmukh

Variables and variable assignments

➢ Global Variables

- If you create a variable with the same name inside a function, this variable will be local, and can only be used inside the function. The global variable with the same name will remain as it was, global and with the original value.
- Create a variable inside a function, with the same name as the global variable

Example:

```
x = "awesome"

def myfunc( ):
  x = "fantastic"
  print("Python is " + x)

myfunc( )

print("Python is " + x)
```

Output:

```
Python is fantastic
Python is awesome
```


© Copyright | Sushama Deshmukh

Variables and variable assignments

➢ The global Keyword

- Normally, when you create a variable inside a function, that variable is local, and can only be used inside that function.
- To create a global variable inside a function, you can use the global keyword.
- If you use the global keyword, the variable belongs to the global scope:

Example:

```
def myfunc( ):
  global x
  x = "fantastic"

myfunc( )

print("Python is " + x)
```

Output:

Python is fantastic

© Copyright | Sushama Deshmukh

Variables and variable assignments

➢ **The global Keyword**

- Also, use the **global** keyword if you want to change a global variable inside a function.

- To change the value of a global variable inside a function, refer to the variable by using the global keyword:

Output:

Python is fantastic

Example:

```
x = "awesome"

def myfunc( ):
  global x
  x = "fantastic"

myfunc( )

print("Python is " + x)
```


© Copyright | Sushama Deshmukh

Knowledge Check



© Copyright | Sushama Deshmukh

Knowledge Check : Exercise 1

Variables and variable assignments



1. Create a variable named carname and assign the value Volvo to it.
2. Create a variable named x and assign the value 50 to it.
3. Display the sum of 5 + 10, using two variables: x and y.
4. Create a variable called z, assign x + y to it, and display the result.
5. Insert the correct syntax to assign values to multiple variables in one line:

   x y z = "Orange", "Banana", "Cherry"
6. Insert the correct syntax to assign the same value to all three variables in one code line.
7. Insert the correct keyword to make the variable x belong to the global scope.

   def myfunc():

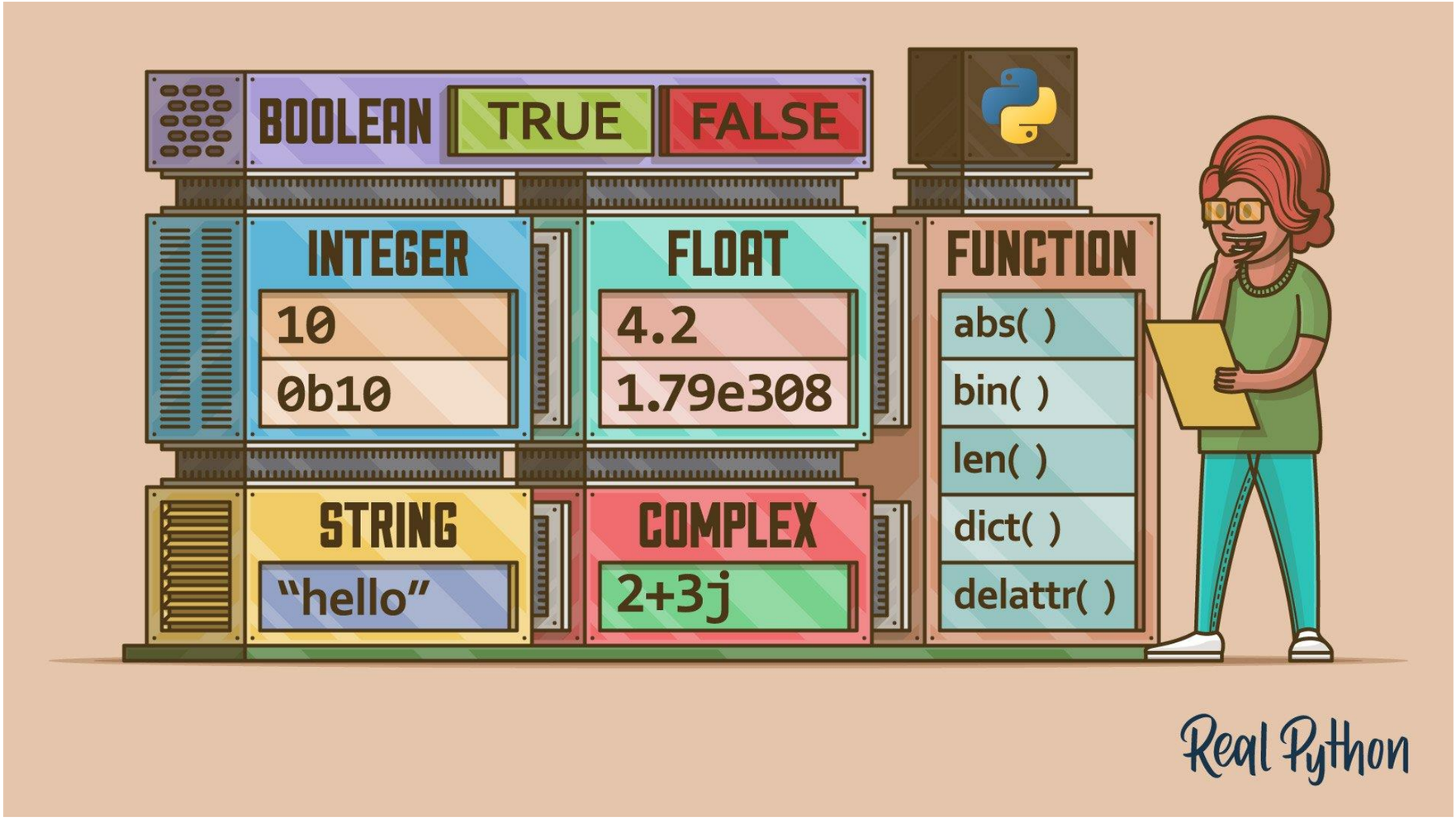
   ------ x

   x = "fantastic"

© Copyright | Sushama Deshmukh

Data Types

- **Built-in Data Types: -** In programming, data type is an important concept.
- Variables can store data of different types, and different types can do different things.



© Copyright | Sushama Deshmukh

Data Types

- **Python has the following data types built-in by default, in these categories:**

Data Type	Keyword
Text Type:	str
Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	dict
Set Types:	set, frozenset
Boolean Type:	bool
Binary Types:	bytes, bytearray, memoryview
None Type:	NoneType

© Copyright | Sushama Deshmukh

Data Types

➢ Getting the Data Type

- You can get the data type of any object by using the type() function:

Example: Print the data type of the variable x:

```
x = 5

print(type(x))
```

➢ Setting the Data Type

- In Python, the data type is set when you assign a value to a variable:

© Copyright | Sushama Deshmukh

Data Types - Setting the Data Type:

- **Setting the Data Type:**
  In Python, **the data type is set when you assign a value to a variable**:

Example	Data Type
x = "Hello World"	str
x = 20	int
x = 20.5	float
x = 1j	complex
x = ["apple", "banana", "cherry"]	list
x = ("apple", "banana", "cherry")	tuple
x = range(6)	range

© Copyright | Sushama Deshmukh

Data Types- Setting the Data Type:

Continued……..

Example	Data Type
x = {"name" : "John", "age" : 36}	dict
x = {"apple", "banana", "cherry"}	set
x = frozenset({"apple", "banana", "cherry"})	frozenset
x = True	bool
x = b"Hello"	bytes
x = bytearray(5)	bytearray
x = memoryview(bytes(5))	memoryview
x = None	NoneType

© Copyright | Sushama Deshmukh

Data Types - Setting the Specific Data Type:

- **Setting the Specific Data Type:** If you want to specify the data type, you can use the following constructor functions:

Example	Data Type
x = str("Hello World")	str
x = int(20)	int
x = float(20.5)	float
x = complex(1j)	complex
x = list(("apple", "banana", "cherry"))	list
x = tuple(("apple", "banana", "cherry"))	tuple
x = range(6)	range

© Copyright | Sushama Deshmukh

Data Types - Setting the Specific Data Type:

Continued……..

Example	Data Type
x = dict(name="John", age=36)	dict
x = set(("apple", "banana", "cherry"))	set
x = frozenset(("apple", "banana", "cherry"))	frozenset
x = bool(5)	bool
x = bytes(5)	bytes
x = bytearray(5)	bytearray
x = memoryview(bytes(5))	memoryview

© Copyright | Sushama Deshmukh

Exercise 2 - Data Types

What will be output of the following code?

Sr. No.	Example	Output ?
1	x = 5 print(type(x))	
2	x = "Hello World" print(type(x))	
3	x = 20.5 print(type(x))	

© Copyright | Sushama Deshmukh

Exercise 2 - Data Types

What will be output of the following code?

Sr. No.	Example	Output ?
4	x = ["apple", "banana", "cherry"] print(type(x))	
5	x = ("apple", "banana", "cherry") print(type(x))	
6	x = {"name" : "John", "age" : 36} print(type(x))	
7	x = True print(type(x))	

© Copyright | Sushama Deshmukh

Exercise 2 - Data Types

Which of these lines of code will give you an error?

Sr. No.	Example	Output/Error ?
A	name = input("What is your name?") print(f"Hello, {name}")	
B	name = input("What is your name?") print("Hello, " + name)	
C	**age = 16** **print(f"You are {age} years old")**	
D	age = 16 print("You are " + age + " years old")	

© Copyright | Sushama Deshmukh

Number Manipulation & F Strings

➢ **How to use f-strings in Python: String Format**

- As we learned in the Python Variables, **we cannot combine strings and numbers** like this:

Example:

```
age = 36

txt = "My name is John, I am " + age

print(txt)
```

Error……

```
Traceback (most recent call last):
  File "demo_string_format_error.py", line 2, in <module>
    txt = "My name is John, I am " + age
TypeError: must be str, not int
```


© Copyright | Sushama Deshmukh

Number Manipulation & F Strings

- But **we can combine strings and numbers by using f-strings or the format() method!**

Example:

```
age = 36

txt = "My name is John, I am " + age

print(txt)
```

Remove error……
By using f-string

Example:

```
age = 36

txt = f"My name is John, I am {age}"

print(txt)
```


© Copyright | Sushama Deshmukh

Number Manipulation & F Strings

➢ **How to use f-strings in Python**

- To create a f-string, **prefix the string with the letter “ f ”**. The string itself can be formatted in much the same way that you would with str.format().
- F-strings provide **a concise and convenient way to embed Python expressions inside string literals** for formatting**.**
- F-string allows you **to format selected parts of a string**.

© Copyright | Sushama Deshmukh

Number Manipulation & F Strings

- **Print Variables using f-string in Python**

Example:

```
val = 'Geeks'
print(f"{val}for{val} is a portal for {val}.")

name = ‘Sushama'
age = 43
print(f"Hello, My name is {name} and I'm {age} years old.")
```

- **O/P:**

GeeksforGeeks is a portal for Geeks.
Hello, My name is Sushama and I'm 43 years old.

© Copyright | Sushama Deshmukh

Number Manipulation & F Strings

- **Print date using f-string in Python**

Example:

```
# Prints today's date with help of datetime library

import datetime

today = datetime.datetime.today()

print(f"{today:%B %d, %Y}")
```

- **O/P:**

June 18, 2024

Note: F-strings are faster than the two most commonly used string formatting mechanisms, which are % formatting and str.format().

© Copyright | Sushama Deshmukh

Number Manipulation & F Strings

- **Evaluate Expressions with f-Strings in Python**

Example:

```
English = 78
Maths = 56
Hindi = 85

print(f"Ram got total marks {English + Maths + Hindi} out of 300")
```

- **O/P:**

Ram got total marks 219 out of 300

© Copyright | Sushama Deshmukh

Placeholders and Modifiers

- To format values in a f-string, add **placeholders { }**, a placeholder can contain **variables, operations, functions, and modifiers to format the value.**

Example: Add a placeholder for the price variable.

```
price = 59
txt = f"The price is {price} dollars"
print(txt)
```

Output:

The price is 59 dollars

© Copyright | Sushama Deshmukh

Placeholders and Modifiers

- A placeholder can also include a modifier to format the value. **A modifier is included by adding a colon :** followed by a legal formatting type, like
  .2f which means fixed point number with 2 decimals.
- You can also format a value directly without keeping it in a variable:

Example: Display the value 95 with 2 decimals:

```
txt = f"The price is {95:.2f} dollars"
print(txt)
```

Example: Display the price with 2 decimals:

```
price = 59
txt = f"The price is {price:.2f} dollars"
print(txt)
```

Output:
The price is 59.00 dollars

Output:
The price is 95.00 dollars

© Copyright | Sushama Deshmukh

Exercise 3 Numbers & F - Strings

Your Life in Weeks

Create a program using maths & f-strings that tells us how many weeks we have left, if we live until 90 years old.

It will take your current age as the input & output a message with our time left in this format:

You have x weeks left.

Where x is replaced with the actual calculated number of weeks the input age has left until age 90.

© Copyright | Sushama Deshmukh

Python Numbers

- **Python Numbers:** There are three numeric types in Python:

  int
  float
  complex

- Variables of numeric types are created when you assign a value to them:

Example:

```
x = 1    # int
y = 2.8  # float
z = 1j   # complex
```


© Copyright | Sushama Deshmukh

Python Numbers

- **To verify the type of any object in Python, use the type() function:**

Example:

```
print(type(x))
print(type(y))
print(type(z))
```

- **Int**

- Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

Example:

Integers:

```
x = 1
y = 35656222554887711
z = -3255522

print(type(x))
print(type(y))
print(type(z))
```


© Copyright | Sushama Deshmukh

Python Numbers

- **Float, or "floating point number"** is a number, positive or negative, containing one or more decimals.

Example:

```
x = 1.10
y = 1.0
z = -35.59

print(type(x))
print(type(y))
print(type(z))
```

- Float can also be scientific numbers with an "e" to indicate the power of 10.

Example:

```
x = 35e3
y = 12E4
z = -87.7e100

print(type(x))
print(type(y))
print(type(z))
```


© Copyright | Sushama Deshmukh

Python Numbers

- **Complex**
- Complex numbers are written with a "j" as the imaginary part:

Example:

```
x = 3+5j
y = 5j
z = -5j

print(type(x))
print(type(y))
print(type(z))
```


© Copyright | Sushama Deshmukh

Python Numbers

➢ **Type Conversion**

- You can convert from one type to another with the int(), float(), and complex() methods:
- **Convert from one type to another** Example:

- **Note:** You cannot convert complex numbers into another number type.

Example:

```
x = 1    # int
y = 2.8  # float
z = 1j   # complex

#convert from int to float:
a = float(x)

#convert from float to int:
b = int(y)

#convert from int to complex:
c = complex(x)

print(a)
print(b)
print(c)

print(type(a))
print(type(b))
print(type(c))
```


© Copyright | Sushama Deshmukh

Python Numbers

➢ Random Number

- Python does not have a random() function to make a random number, but Python has a built-in module called random that can be used to make random numbers:

- **Example:** Import the random module, and display a random number between 1 and 9:

Example:

```
import random
print(random.randrange(1, 10))
```


© Copyright | Sushama Deshmukh

Exercise 3 - Python Numbers

1. **Insert the correct syntax to convert x into a floating point number.**

```
x = 5
x =    (x)
```

2. **Insert the correct syntax to convert x into an integer.**

```
x = 5.5
x =    (x)
```

3. **Insert the correct syntax to convert x into a complex number.**

```
x = 5
x =    (x)
```


© Copyright | Sushama Deshmukh

Python Operators

- Operators are used to perform operations on variables and values.
- In the example below, we use the + operator to add together two values:

Example:

```
print(10 + 5)
```

- **Python divides the operators in the following groups:**
  - Arithmetic operators
  - Assignment operators
  - Comparison operators
  - Logical operators
  - Identity operators
  - Membership operators
  - Bitwise operators

© Copyright | Sushama Deshmukh

Python Operators

- **Arithmetic Operators:** Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

© Copyright | Sushama Deshmukh

Python Operators

- **Assignment Operators:** Assignment operators are used to assign values to variables:

Operator	Example	Same as
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3

© Copyright | Sushama Deshmukh

Python Operators

- **Assignment operators continued…**

Operator	Example	Same as
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3
:=	print(x := 3)	x = 3 print(x)

© Copyright | Sushama Deshmukh

Python Operators

- **Comparison Operators:** Comparison operators are used to compare two values:

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

© Copyright | Sushama Deshmukh

Python Operators

- **Logical Operators:** are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

© Copyright | Sushama Deshmukh

Python Operators

- **Identity Operators:** are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

© Copyright | Sushama Deshmukh

Python Operators

- **Membership Operators:** are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

© Copyright | Sushama Deshmukh

Python Operators

- **Bitwise Operators:** are used to compare (binary) numbers:

Operator	Name	Description	Example
&	AND	Sets each bit to 1 if both bits are 1	x & y
|	OR	Sets each bit to 1 if one of two bits is 1	x | y
^	XOR	Sets each bit to 1 if only one of two bits is 1	x ^ y
~	NOT	Inverts all the bits	~x
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off	x << 2
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	x >> 2

© Copyright | Sushama Deshmukh

Python Operators

- **Operator Precedence:** describes the order in which operations are performed.

Example:

- Parentheses has the highest precedence, meaning that expressions inside parentheses must be evaluated first:

```
print((6 + 3) - (6 + 3))
```

- Multiplication * has higher precedence than addition +, and therefor multiplications are evaluated before additions:

```
print(100 + 5 * 3)
```


© Copyright | Sushama Deshmukh

Python Operators

- **The precedence order** is described in the table below, starting with the highest precedence at the top:

Operator	Description
( )	Parentheses
**	Exponentiation
+x, -x, ~x	Unary plus, unary minus, and bitwise NOT
*, /, //, %	Multiplication, division, floor division, and modulus
+ -	Addition and subtraction
<< >>	Bitwise left and right shifts
&	Bitwise AND

© Copyright | Sushama Deshmukh

Python Operators

- **The precedence order** continued……

Operator	Description
^	Bitwise XOR
|	Bitwise OR
== != > >= < <= is is not in not in	Comparisons, identity, and membership operators
not	Logical NOT
and	AND
or	OR
^	Bitwise XOR

© Copyright | Sushama Deshmukh

Python Operators

- **Precedence Order:** If two operators have the same precedence, the expression is evaluated from left to right.
  Example:

- **Addition + and subtraction - has the same precedence,** and therefor we **evaluate the expression from left to right:**

```
print(5 + 4 - 7 + 3)
```

- **Parentheses has the highest precedence, meaning that expressions inside parentheses must be evaluated first:**

© Copyright | Sushama Deshmukh

Exercise 4 - Python Operators

1. Multiply 10 with 5, and print the result.

2. Divide 10 by 2, and print the result.

3. Use the correct membership operator to check if "apple" is present in the fruits object.

```
fruits = ["apple", "banana"]
if "apple"  .....   fruits:
   print("Yes, apple is a fruit!")
```

Floating Point Arithmetic: Issues and Limitations
https://docs.python.org/3/tutorial/floatingpoint.html

© Copyright | Sushama Deshmukh

Exercise 4 - Python Operators

4. Use the correct comparison operator to check if 5 is not equal to 10.

```
if 5.... 10:
  print("5 and 10 is not equal")
```

5. Use the correct logical operator to check if at least one of two statements is True.

```
if 5 == 10.....4 == 4:
  print("At least one of the statements is true")
```


© Copyright | Sushama Deshmukh

Exercise 4 - Python Operators

6. You are a computer. What will this line of code print?

```
(6 + 4/2 - (1*2))
```

7. What is the data type of the result of the variable a in the following line of code:

```
a = int("5") / int(2.7)
```


© Copyright | Sushama Deshmukh

Small Project
Tip Calculator

Create a Tip Calculator, that takes total bill as input with 12 % tip. The 12% tip should be divided equally between the number of people.

Output:

Welcome to the Tip calculator!
What was the total bill? $150.00
How much tip would you like to give? 10, 12, or 15? 12
How many people to split the bill? 5
Each person should pay: $33.60

© Copyright | Sushama Deshmukh

Reference Books

- **Text Books:**

1. D. Lee, “Python Programming Fundamentals”, Second Edition, Springer Publication..
2. Wes McKinney, “Python for Data Analysis” O’Reilly Publication.

- **Reference Books:**

1. The Python Language Reference: http://docs.python.org/2/reference/index.html
2. The Python Standard Library: http://docs.python.org/2/library/
3. https://docs.scipy.org/doc/scipy/reference/tutorial/stats.html
4. http://matplotlib.org/api/mlab_api.html#module-matplotlib.mlab
5. http://conference.scipy.org/proceedings/scipy2010/pdfs/seabold.pdf
6. http://seaborn.pydata.org
7. https://www.datacamp.com/community/data-science-cheatsheets
8. PEP 20 -- The Zen of Python: https://www.python.org/dev/peps/pep-0020/
9. https://docs.scipy.org/doc/numpy-dev/user/numpy-for-matlab-users.html

© Copyright | Sushama Deshmukh

Web References

https://www.w3schools.com/python/python_getstarted.asp
https://www.turing.com/kb/how-to-use-jupyter-notebook
https://www.tutorialspoint.com/python/python_interpreter.htm
https://www.geeksforgeeks.org/input-and-output-in-python/

Programming for Everybody (Getting Started with Python)

This course is part of Python for Everybody Specialization
Taught in English
Some content may not be translated



Instructor: Charles Russell Severance

© Copyright | Sushama Deshmukh

Thank You

© Copyright | Sushama Deshmukh